

Módulo V

Multimedia

Audio · Camara · Video

- 1 — Permisos en runtime — microfono y camara
- 2 — MediaPlayer — reproducción de audio
- 3 — MediaRecorder — grabación de audio
- 4 — Camara con Intent — fotos y FileProvider
- 5 — Video — grabación y reproducción

Permisos en runtime

Para usar micrófono y cámara Android exige declararlos en el Manifest y solicitarlos al usuario en tiempo de ejecución:

AndroidManifest.xml:

```
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="28"/>
```

Solicitar en la Activity:

```
private val permLauncher =
    registerForActivityResult(
        RequestPermission()) { granted ->
        if (granted) iniciarGrabacion()
        else Toast.makeText(this,
            "Permiso denegado",
            Toast.LENGTH_SHORT).show()
        }

fun verificarPermiso() {
    val p = Manifest.permission.RECORD_AUDIO
    if (checkSelfPermission(p) ==
        PERMISSION_GRANTED) {
        iniciarGrabacion()
    } else {
        permLauncher.launch(p)
    }
}
```

Desde Android 6.0 (API 23): los permisos peligrosos (micrófono, cámara, almacenamiento) deben solicitarse en runtime. Solo declararlos en el Manifest no es suficiente.

MediaPlayer — reproducción de audio

MediaPlayer reproduce audio desde recursos locales, archivos o URLs:

```
// Desde res/raw/mi_audio.mp3
var player: MediaPlayer? = null    override fun onStop() {
                                   super.onStop()
                                   detener()           // liberar en
fun reproducir() {                  ciclo
    player = MediaPlayer.create(    this, R.raw.mi_audio    }
    this, R.raw.mi_audio          )
    player?.start()
}

fun pausar()    { player?.pause()
}
fun reanudar() { player?.start()
}

fun detener() {
    player?.apply {
        stop()
        release()    // SIEMPRE
    }
    player = null
}
```

```
// Desde archivo en almacenamiento          "Error de audio",
fun reproducirArchivo(ruta:                Toast.LENGTH_SHORT).show()
String) {                                  true
    player?.release()                      }
    player = MediaPlayer().apply {
        setDataSource(ruta)
        prepare()    // sincrónico
        start()
    }
}

// Callback al terminar
player?.setOnCompletionListener {
    btnPlay.text = "Reproducir"
    it.release()
    player = null
}

// Error al cargar el archivo
player?.setOnErrorListener {
    _'_'_>
    Toast.makeText(this,
```

release() es obligatorio: sin el, el proceso de audio sigue activo en segundo plano consumiendo memoria y batería, aunque el usuario haya cerrado la pantalla.

MediaRecorder — grabación de audio

MediaRecorder graba audio del micrófono y lo guarda en un archivo. Requiere permiso RECORD_AUDIO:

```
private var recorder: MediaRecorder? = null
private var rutaAudio = ""

fun iniciarGrabacion() {
    rutaAudio = "${externalCacheDir
        ?.absolutePath}/audio_${
            System.currentTimeMillis()}.mp4"

    recorder = MediaRecorder(this).apply {
        // Orden estricto — no cambiar
        setAudioSource(
            MediaRecorder.AudioSource.MIC)
        setOutputFormat(
            MediaRecorder.OutputFormat.MPEG_4)
        setAudioEncoder(
            MediaRecorder.AudioEncoder.AAC)
        setOutputFile(rutaAudio)
        prepare()
        start()
    }
}
```

```
fun detenerGrabacion() {
    recorder?.apply {
        stop()
        release()
    }
    recorder = null
    // rutaAudio tiene la ruta
    // del archivo grabado
}

override fun onStop() {
    super.onStop()
    recorder?.apply {
        stop(); release()
    }
    recorder = null
}

// Guardar la ruta en la BD
// para reproducir despues
vm.guardarNota(
    Nota(rutaAudio = rutaAudio)
)
```

Orden obligatorio: setAudioSource → setOutputFormat → setAudioEncoder → setOutputFile → prepare() → start(). Cambiar el orden lanza IllegalStateException.

Rutas de almacenamiento para archivos multimedia

Según el caso de uso se elige entre cache privada, almacenamiento privado persistente o almacenamiento publico:

Ubicación	Metodo	Persiste	Visible	Permiso API 29+
Cache privada	externalCacheDir	No	No	No necesita
Archivos privados	getExternalFilesDir()	Si	No	No necesita
Almacenamiento publico	MediaStore (API 29+)	Si	Si	No necesita
Almacenamiento publico	Environment (API 28-)	Si	Si	WRITE_EXTERNAL_STORAGE

Ruta recomendada para el curso:

```
// Audio
val rutaAudio = "${getExternalFilesDir(Environment.DIRECTORY_MUSIC)}/audio_${System.currentTimeMillis()}.mp4"
// Fotos
val rutaFoto = "${getExternalFilesDir(Environment.DIRECTORY_PICTURES)}/foto_${System.currentTimeMillis()}.jpg"
// Video
val rutaVideo = "${getExternalFilesDir(Environment.DIRECTORY_MOVIES)}/video_${System.currentTimeMillis()}.mp4"
```

Tomar fotos con TakePicture()

TakePicture() delega la captura a la app de cámara del sistema. Es la forma más sencilla y no requiere permiso CAMERA:

```
private val tomarFotoLauncher =
    registerForActivityResult(
        TakePicture()) { exito ->
        if (exito) mostrarFoto(fotoUri)
    }

private var fotoUri: Uri? = null

fun abrirCamara() {
    // Crear el archivo ANTES
    // de lanzar el launcher
    fotoUri = crearArchivoFoto()
    tomarFotoLauncher.launch(fotoUri)
}

fun mostrarFoto(uri: Uri?) {
    uri ?: return
    imgFoto.setImageURI(uri)
}
```

```
fun crearArchivoFoto(): Uri {
    val dir = getExternalFilesDir(
        Environment.DIRECTORY_PICTURES)
    val archivo = File(
        dir,
        "foto_${System
            .currentTimeMillis()}.jpg"
    )
    // FileProvider convierte
    // file:// → content://
    return FileProvider.getUriForFile(
        this,
        "${packageName}.fileprovider",
        archivo
    )
}
```

Siempre crear la URI ANTES de lanzar el launcher. Si **fotoUri** es **null** cuando la cámara retorna el resultado, la foto no se puede mostrar, aunque haya sido tomada correctamente.

FileProvider — URIs seguras entre apps

FileProvider es obligatorio desde Android 7.0. Convierte rutas de archivo en URIs content:// seguras:

AndroidManifest.xml — agregar dentro de <application>:

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data android:name="android.support.FILE_PROVIDER_PATHS" android:resource="@xml/file_paths"/>
</provider>
```

res/xml/file_paths.xml — crear este archivo:

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
    <external-files-path
        name="pictures"
        path="Pictures"/>
    <external-files-path
        name="movies"
        path="Movies"/>
</paths>
```

El resultado:

```
// SIN FileProvider — FALLA en API 24+
// file:///storage/.../foto.jpg

// CON FileProvider — funciona
// content://com.example.fileprovider
//           /pictures/foto.jpg

// FileNotFoundException si
// usas file:// con otra app
```

Error común: olvidar crear res/xml/file_paths.xml o no declarar el provider en el Manifest. Ambos causan un crash inmediato al intentar tomar una foto.

Mostrar y cargar imágenes

Dos opciones para mostrar fotos — Glide (con librería) y BitmapFactory (sin librería):

Glide — recomendado:

```
// build.gradle
implementation "com.github.bumptech
    .glide:glide:4.16.0"

// Desde URI
Glide.with(this)
    .load(fotoUri)
    .placeholder(R.drawable.ic_foto)
    .error(R.drawable.ic_error)
    .centerCrop()
    .into(imgFoto)

// Desde ruta de archivo
Glide.with(this)
    .load(File(rutaFoto))
    .into(imgFoto)
```

BitmapFactory — sin dependencias:

```
fun cargarFoto(ruta: String): Bitmap? {
    // inSampleSize=2: imagen a 1/4
    // de la memoria original
    val opts = BitmapFactory
        .Options().apply {
            inSampleSize = 2
        }
    return BitmapFactory
        .decodeFile(ruta, opts)
}

// Usar en un hilo de fondo
// para no bloquear la UI
viewModelScope.launch {
    val bmp = withContext(IO) {
        cargarFoto(ruta)
    }
    imgFoto.setImageBitmap(bmp)
}
```

Glide vs BitmapFactory: Glide maneja caching, redimensionado y carga asíncrona automáticamente. BitmapFactory puede causar OutOfMemoryError si no se usa inSampleSize. Para el curso, Glide es la opción mas practica.

Grabar video con CaptureVideo()

CaptureVideo() delega la grabación a la app de cámara del sistema. El patrón es idéntico al de TakePicture():

```
private val grabarVideoLauncher =
    registerForActivityResult(
        Environment.DIRECTORY_MOVIES),
        CaptureVideo()) { éxito ->
            if (éxito)
reproducirVideo(videoUri)                .currentTimeMillis() }.mp4"
    }
        return

private var videoUri: Uri? = null FileProvider.getUriForFile(
    this,

fun abrirCamaraVideo() {
    videoUri =
    crearArchivoVideo()                archivo

grabarVideoLauncher.launch(videoU }
ri)
}

fun crearArchivoVideo(): Uri {
    val archivo = File(
        getExternalFilesDir(
```

```
// Guardar la ruta en la BD
// y reproducir en VideoView
fun reproducirVideo(uri: Uri?) {
    uri ?: return
    val mc = MediaController(this)
    mc.setAnchorView(videoView)
    with(videoView) {
        setMediaController(mc)
        setVideoURI(uri)
        requestFocus()
        start()
    }
}

// Guardar en BD via ViewModel
vm.guardarNota(Nota(
    titulo = "Video ${Date()}",
    rutaVideo =
videoUri.toString()
))
```

Mismo patrón que TakePicture(): crear la URI con FileProvider ANTES de lanzar, recibir el resultado en el callback. El archivo queda en getExternalFilesDir(MOVIES).

VideoView — reproducción de video

VideoView es el componente para mostrar video en la UI. Soporta URIs locales, rutas de archivo y URLs:

Layout XML:

```
<VideoView
    android:id="@+id/videoView"
    android:layout_width=
        "match_parent"
    android:layout_height=
        "220dp"
    android:layout_marginTop="8dp"/>
```

Kotlin — con MediaController:

```
val videoView = findViewById<VideoView>(R.id.videoView)
                                Toast.makeText(this, "Video terminado",
                                                Toast.LENGTH_SHORT).show()

fun reproducirVideo(uri: Uri) {
    // MediaController: botones
    // play / pause / seek
    val mc = MediaController(this) override fun onStop() {
        mc.setAnchorView(videoView)
        super.onStop()
        videoView.stopPlayback()
    }
    videoView.setMediaController(mc)

    videoView.setVideoURI(uri)
    videoView.requestFocus()
    videoView.start()

    videoView
        .setOnCompletionListener {
```

stopPlayback() en onStop(): libera los recursos del VideoView cuando la Activity va a segundo plano. Sin esto el video sigue decodificando aunque no sea visible.

MVVM aplicado a multimedia

Las rutas de archivos se guardan en la BD como Strings. El ViewModel gestiona el estado sin tocar la UI directamente:

```
// Entity — rutas como Strings
@Entity(tableName = "notas")
data class Nota(
    val titulo: String,
    val rutaAudio: String? = null,
    val rutaFoto: String? = null,
    val rutaVideo: String? = null,
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0
)

private val _notas =
    MutableStateFlow<List<Nota>>(emptyList())
    val notas =
        _notas.asStateFlow()

fun guardar(nota: Nota) {
    viewModelScope.launch {
        repo.insertar(nota)
        _notas.value =
            repo.getAll()
    }
}

// ViewModel
class NotaViewModel(app: Application)
    : AndroidViewModel(app) {
    private val repo =
        NotaRepository(app)
}
```

```
// Activity — solo captura y observa
private val fotoLauncher =
    registerForActivityResult(
        TakePicture()) { éxito ->
        if (éxito) {
            vm.guardar(Nota(
                titulo = "Foto ${Date()}",
                rutaFoto =
                    fotoUri.toString()
            ))
        }
    }

// Observar la lista
lifecycleScope.launch {
    vm.notas.collect { lista ->
        adapter.actualizar(lista)
    }
}
```

Las rutas son Strings, no archivos. La BD guarda la ruta del archivo. El archivo vive en el sistema de archivos del dispositivo y se carga con Glide o VideoView cuando se necesita mostrar.

Errores comunes en multimedia

Los errores mas frecuentes al trabajar con audio, cámara y video:

FileUriExposedException

Causa: Pasar file:// a otra app en API 24+

Fix: Usar `FileProvider.getUriForFile()`

PERMISSION_DENIED al grabar

Causa: No se verifico RECORD_AUDIO antes de grabar

Fix: `checkSelfPermission()` antes de `MediaRecorder.start()`

IllegalStateException

Causa: Orden incorrecto en la configuración del MediaRecorder

Fix: `setSource` → `setFormat` → `setEncoder` → `setOutputFile` → `prepare()` → `start()`

OutOfMemoryError con imágenes

Causa: Cargar foto de alta resolución sin redimensionar

Fix: Usar `Glide` o `BitmapFactory.Options` con `inSampleSize`

Foto en blanco — URI null

Causa: `fotoUri` era `null` al lanzar el launcher

Fix: Crear la URI con `crearArchivoFoto()` ANTES de `launch()`

Video no se reproduce

Causa: `start()` llamado antes de que el video cargue

Fix: `setVideoURI` + `start()` son suficientes — `VideoView` llama `prepare()` interno

Fuga de memoria — MediaPlayer

Causa: No se llama `release()` al terminar

Fix: `release()` en `onStop()` o en `setOnCompleteListener`

Componentes multimedia — cuando usar cada uno

Guía de selección del componente correcto según el caso de uso:

Componente	Para que sirve	Cuando usarlo	Limitación
MediaPlayer	Reproducir audio y video	Reproducción desde archivos o recursos locales	No graba. release() obligatorio
MediaRecorder	Grabar audio y video	Control total sobre formato y calidad de grabación	Configuración en orden estricto
TakePicture()	Tomar fotos con la cámara del sistema	Forma rápida sin implementar cámara propia	Menos control sobre la cámara
CaptureVideo()	Grabar video con la cámara del sistema	Forma rápida sin implementar grabación propia	Menos control sobre duración y calidad
VideoView	Reproducir video en la UI	Mostrar video local o desde URL en un View	Sin acceso al decodificador interno
FileProvider	Compartir archivos con otras apps	Siempre que se pase una URI a TakePicture o CaptureVideo	Requiere Manifest + xml/file_paths.xml

Módulo V — Resumen

1

Permisos en runtime

RECORD_AUDIO y CAMERA desde API 23. Declarar en Manifest y solicitar con RequestPermission() en tiempo de ejecución.

3

MediaRecorder

Graba audio. Orden: setSource → setFormat → setEncoder → setOutputFile → prepare() → start(). release() al detener.

5

VideoView

Reproduce video en la UI. MediaController para controles de play/pause. stopPlayback() en onStop().

2

MediaPlayer

Reproduce audio y video. release() obligatorio en onStop(). setOnCompletionListener para detectar cuando termino.

4

Camara y FileProvider

TakePicture() y CaptureVideo() con URI de FileProvider. Crear la URI ANTES de lanzar el launcher.

6

MVVM con multimedia

Las rutas de archivos son Strings en la BD. El ViewModel guarda y carga sin que la Activity toque el sistema de archivos.